



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



INSTITUTO TECNOLÓGICO DE CHETUMAL

UNIDAD 3. PROGRAMACIÓN

Sistema de Gestión del Servicio Social

VERSIÓN:

“1.0”

INGENIERÍA DE SOFTWARE

REALIZADO PARA:

TECNM Campus Chetumal (ItChetumal)

REALIZADO POR

COLLÍ KUMUL SAÚL MISAEAL

MEDINA MAGLAH NOÉ ALEJANDRO

PUC ESPENS URIEL

TUN ROBERTO ANGEL

DOCENTE:

ZAPATA ENCALADA MANUEL ABRAHAM

CHETUMAL, QUINTANA ROO, MAYO DE 2026

Contenido

1. Lenguaje de desarrollo del sistema	3
1.1 Descripción del Equipo de Trabajo.....	3
1.2 Criterios para la Selección del Lenguaje de Programación	5
1.3 Lenguaje de Programación Seleccionado	6
2. Manejador de base de datos	7
2.1 Descripción del Manejador de Base de Datos a Utilizar	7
2.2 Diseño Lógico	9
2.3 Diseño Físico	10
3. Construcción del sistema	10
3.1 Definición del Estilo de Programación	10
3.2 Código de Programación del Proyecto	12

1. LENGUAJE DE DESARROLLO DEL SISTEMA

1.1 Descripción del Equipo de Trabajo

Saúl Misael Collí Kumul — Líder de Proyecto

Asume la máxima responsabilidad estratégica y organizativa del proyecto en el rol de Líder de Proyecto. Su gestión se enfoca en coordinar el ciclo de vida completo del desarrollo del software, asegurando que los objetivos de digitalización mitiguen de forma efectiva la carga de trabajo repetitiva del personal administrativo. Actúa como el canal de comunicación formal y enlace institucional entre el equipo de desarrollo técnico y los diversos actores del Instituto Tecnológico de Chetumal, lo que incluye al Departamento de Gestión Tecnológica y Vinculación, a la comunidad estudiantil aspirante y a las organizaciones externas.

Además, se ha encargado directamente de la planeación y el control del catálogo de requisitos, definiendo las pautas de usabilidad para evitar la resistencia al cambio tecnológico por parte de usuarios externos. Su trabajo técnico se concentró en la elicitación, redacción y modelado iterativo de los diagramas de casos de uso globales y en el establecimiento de las reglas de negocio. Como responsable de la calidad documental, coordinó y validó las correcciones integrales que dieron origen a las versiones del documento de requisitos del sistema.

Noé Alejandro Medina Maglah — Desarrollador de Software

El encargado de la construcción lógica y la programación en el servidor, desempeñándose como el Desarrollador Backend del equipo. Su labor principal consiste en traducir las especificaciones de requisitos y los flujos abstractos descritos en la documentación hacia un entorno de código totalmente ejecutable. Para ello, utiliza la infraestructura de la arquitectura cliente-servidor basada en el paquete monolítico XAMPP, implementando scripts estructurados en lenguaje PHP y enlazándolos de manera segura con el motor relacional de la base de datos.

Entre sus responsabilidades operacionales se encuentra el desarrollo del controlador o enrutador principal del sistema. Asimismo, es el programador responsable de codificar el sistema transaccional de carga de archivos binarios, implementando validaciones estrictas en el servidor para restringir el tamaño y la extensión de los documentos PDF cargados por los alumnos. En la documentación del proyecto, su aporte técnico consistió en la especificación detallada en pseudocódigo del módulo de acceso, el subsistema de inscripciones y la lógica secuencial con sus flujos alternos para el procesamiento y recepción del plan de trabajo.

Uriel Puc Espens — Analista de Datos y Requisitos

Es el especialista responsable de estudiar los procesos actuales e inspeccionar la calidad de la información del sistema. Su trabajo inicial consistió en realizar un análisis del flujo procedimental manual y de los cuellos de botella existentes en las Oficinas de Servicio Social, Residencias Profesionales y Desarrollo Comunitario, fundamentando su investigación en el Manual de Organización Institucional de la SEP y los lineamientos del TecNM. A partir de este diagnóstico, se encarga de modelar los requerimientos funcionales y no funcionales orientados a mantener la consistencia del software.

Su enfoque principal dentro del diseño técnico es asegurar que el sistema impida fallos críticos como la duplicidad de registros escolares o la pérdida de horas acumuladas por los estudiantes durante sus actividades. Adicionalmente, se encarga de verificar que la plataforma responda con precisión a las normativas de identidad institucional mediante el análisis de los requerimientos para la generación automática de la reportería digital oficial. Sus entregables documentales directos en este proyecto abarcan la redacción analítica de la estructura del negocio actual, la delimitación del alcance operativo del software y la elaboración del glosario formal de términos.

Roberto Angel Tun — Arquitecto de Software y Administrador de Base de Datos

Es el responsable directo de la infraestructura tecnológica y el almacenamiento persistente del proyecto. Su función principal consiste en diseñar la topología física y lógica de la solución de software, adoptando formalmente el patrón cliente-servidor de dos capas para garantizar una disponibilidad continua del servicio y tiempos de respuesta óptimos bajo el entorno institucional. En el ámbito de los datos, se encargó del diseño, estructuración y normalización del esquema físico relacional de la base de datos denominado `bd_servicio_social`.

Para garantizar la integridad referencial del sistema, configuró el motor de almacenamiento InnoDB y definió el mapeo estricto de las llaves primarias y foráneas entre las tablas de usuarios, alumnos, dependencias, programas, solicitudes, documentos y evaluaciones. De manera complementaria, programó los módulos utilitarios e integró los servicios externos necesarios para el funcionamiento global del software, tales como el enlace SMTP con el servidor de correo oficial del tecnológico para el envío de notificaciones automatizadas y el uso de librerías para el renderizado de reportes PDF. Sus entregables técnicos incluyen los scripts de creación de la base de datos, el diagrama de estructura modular y el plan de dimensionamiento de hardware para el servidor.

1.2 Criterios para la Selección del Lenguaje de Programación

a) Compatibilidad con la infraestructura institucional y el entorno de despliegue

El criterio primordial para la selección del lenguaje fue el acoplamiento directo con la infraestructura tecnológica preexistente y el entorno físico del servidor del instituto. Al haberse dictaminado el uso del paquete monolítico XAMPP sobre un sistema operativo de servidor compatible, PHP se posicionó como la opción nativa idónea. Este lenguaje elimina la necesidad de configuraciones complejas de variables de entorno o servidores de aplicaciones dedicados que requerirían alternativas como Java o Node.js. PHP se integra de forma inmediata con el servidor web Apache y el motor de bases de datos MySQL/MariaDB a través de extensiones nativas estables sin generar costos adicionales de licenciamiento o modificaciones drásticas en la topología de red existente.

b) Facilidad para el manejo y procesamiento de archivos binarios

Un requerimiento funcional crítico del sistema es la gestión de la documentación de los alumnos, la cual exige que la plataforma sea capaz de recibir, validar y almacenar archivos en formato PDF de hasta 5MB (como planes de trabajo y reportes bimestrales). El lenguaje seleccionado debía ofrecer un manejo de flujos de archivos que fuera eficiente y seguro en el lado del servidor. PHP cumple con este criterio de manera sobresaliente al integrar directivas de configuración nativas y funciones globales orientadas a la verificación de tipos MIME, control de tamaños de carga y movimiento seguro de archivos temporales hacia el almacenamiento físico en disco. Esto facilita al desarrollador backend la implementación de validaciones estrictas que mitigan vulnerabilidades comunes de inyección de archivos sin penalizar el rendimiento del servidor.

c) Madurez del ecosistema y soporte para librerías requeridas

El sistema exige capacidades específicas como el envío de notificaciones automatizadas por correo electrónico y la generación de reportes e informes institucionales descargables en formato PDF. Por lo tanto, el lenguaje elegido debía contar con un ecosistema de librerías maduro, estable y de código abierto. PHP destaca en este aspecto al ofrecer soluciones probadas y ampliamente documentadas, tales como la extensión PHPMailer para la conexión segura mediante el protocolo SMTP institucional (mail.itchetumal.edu.mx), así como las librerías FPDF y TCPDF para la construcción dinámica de documentos desde el backend. La existencia de estas herramientas maduras evitó que el equipo tuviera que programar módulos de renderizado desde cero, asegurando la confiabilidad en la emisión de las cartas de aceptación y constancias de liberación.

d) Consistencia en la persistencia de datos y acoplamiento con MySQL

Dado que el diseño del sistema descansa sobre una base de datos relacional robusta

(bd_servicio_social) configurada bajo el motor InnoDB, el lenguaje de programación debía ofrecer una capa de abstracción de datos eficiente y segura para interactuar con la base de datos. PHP proporciona la extensión nativa PDO (PHP Data Objects), la cual cumple plenamente con este criterio al permitir la ejecución de consultas preparadas. Esta característica es vital para el proyecto por dos razones: en primer lugar, garantiza la seguridad del sistema al prevenir ataques de inyección SQL en formularios críticos como el inicio de sesión; en segundo lugar, facilita la ejecución de transacciones complejas que aseguran la consistencia e integridad de la información escolar del alumno al momento de actualizar sus horas acumuladas o registrar evaluaciones.

1.3 Lenguaje de Programación Seleccionado

a) Paradigma y naturaleza del lenguaje

PHP (Hypertext Preprocessor) es un lenguaje de programación de código abierto, interpretado y de propósito general, cuyo diseño está orientado principalmente al desarrollo backend y a la creación de aplicaciones web dinámicas. Una de sus principales virtudes arquitectónicas es su capacidad para ejecutarse enteramente en el lado del servidor, procesando la lógica de negocio, las consultas a la base de datos y las restricciones de seguridad antes de renderizar y enviar un resultado en formato HTML limpio hacia el navegador del cliente. Además, se define como un lenguaje de tipado dinámico y flexible que soporta múltiples paradigmas de programación, destacando su compatibilidad madura con la Programación Orientada a Objetos (POO). Esto último permite al equipo segmentar el software del servicio social en componentes o módulos lógicos independientes y reutilizables, facilitando el mantenimiento y la escalabilidad del sistema monolítico bajo el entorno de desarrollo propuesto.

b) Modelo de ejecución y arquitectura en el servidor

El funcionamiento de PHP se basa en un modelo de ciclo de vida de solicitud y respuesta sumamente eficiente para plataformas transaccionales. Cuando un usuario (ya sea un alumno, personal de la DGTyV o una dependencia externa) interactúa con la interfaz web y realiza una acción, la petición HTTP es interceptada en el servidor por el software de Apache incluido en el paquete XAMPP. El servidor web delega la ejecución de los scripts al intérprete de PHP, el cual procesa secuencialmente las líneas de código, evalúa las reglas de negocio, se comunica de forma persistente con el motor relacional de la base de datos y compila dinámicamente la salida. Este flujo concluye de manera síncrona cuando el servidor devuelve la respuesta al cliente, liberando inmediatamente los recursos de memoria asignados a esa petición, lo que proporciona al sistema una excelente capacidad de respuesta para manejar

múltiples solicitudes de validación de expedientes de forma simultánea.

c) Mecanismos de seguridad y mitigación de vulnerabilidades

En el contexto de la gestión de información escolar y datos confidenciales, PHP ofrece un conjunto robusto de herramientas nativas diseñadas para proteger la integridad del sistema contra las vulnerabilidades web más comunes. Para mitigar los riesgos de inyección SQL en formularios críticos como el inicio de sesión o la asignación de calificaciones, el lenguaje incorpora la interfaz de objetos de datos de PHP (PDO), la cual obliga al uso de consultas preparadas y la parametrización estricta de variables. Adicionalmente, el lenguaje cuenta con funciones criptográficas avanzadas de última generación para el procesamiento hash y la verificación de contraseñas de manera irreversible en la base de datos, cumpliendo plenamente con las restricciones de seguridad exigidas para la auditoría de expedientes digitales.

d) Capacidades de integración externa y procesamiento de recursos

El lenguaje destaca por su amplia versatilidad para interactuar con sistemas de archivos del servidor y protocolos de red externos mediante componentes modulares de fácil implementación. PHP maneja de forma nativa flujos de archivos que permiten automatizar de manera segura la carga, validación y almacenamiento de los documentos en formato PDF requeridos para el expediente estudiantil. Por otro lado, su ecosistema provee una excelente compatibilidad con extensiones y librerías externas de código abierto indispensables para los requerimientos no funcionales del proyecto. Entre ellas sobresalen las herramientas orientadas a establecer conexiones seguras a través de sockets TCP/IP utilizando el protocolo SMTP para el envío de alertas institucionales automatizadas, así como motores dedicados al procesamiento de gráficos y estructuración de capas vectoriales que facilitan el renderizado y la exportación de documentos oficiales del TecNM, como las constancias finales de liberación de servicio social.

2. MANEJADOR DE BASE DE DATOS

2.1 Descripción del Manejador de Base de Datos a Utilizar

a) Modelo relacional y cumplimiento de propiedades ACID

MySQL/MariaDB es un sistema manejador de bases de datos basado en el modelo relacional, lo que significa que organiza la información en tablas estructuradas mediante filas y columnas,

conectadas entre sí a través de relaciones lógicas estrictas. Técnicamente, su mayor fortaleza para este proyecto radica en su soporte pleno para las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) a través del motor de almacenamiento transaccional InnoDB. Esta característica garantiza que cualquier operación crítica en el servicio social (como la inscripción de un alumno, la carga de un documento o la acumulación de horas bimestrales) se ejecute de forma atómica, es decir, o se completa por entero o no se aplica ningún cambio. Esto previene la corrupción de datos, asegura la consistencia de los registros escolares y garantiza que la información permanezca inalterable ante fallos del servidor.

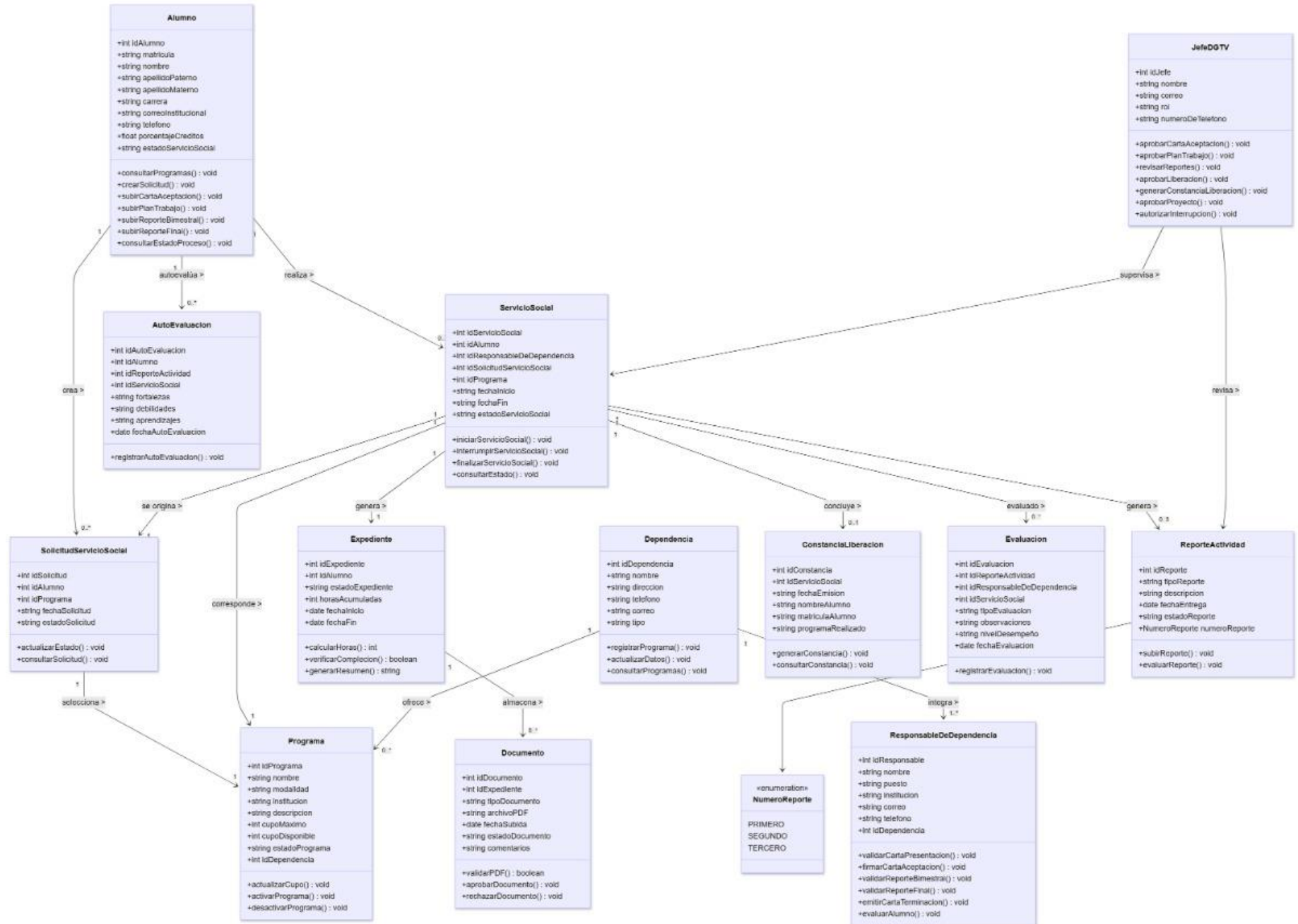
b) Arquitectura multihilo y alta disponibilidad

Desde el punto de vista del rendimiento del servidor, el gestor opera bajo una arquitectura cliente-servidor nativa y utiliza un modelo de ejecución multihilo. Esto significa que cada vez que un usuario realiza una consulta en la plataforma, el manejador asigna un hilo de procesamiento independiente para atender la solicitud. Esta capacidad técnica permite al sistema procesar múltiples peticiones de manera simultánea sin bloquear el servidor web, ofreciendo los tiempos de respuesta de 3 a 5 segundos exigidos en los requerimientos. Adicionalmente, cuenta con mecanismos avanzados de gestión de memoria caché para consultas frecuentes e índices optimizados, lo que reduce drásticamente el uso de CPU y de disco en el servidor institucional del plantel.

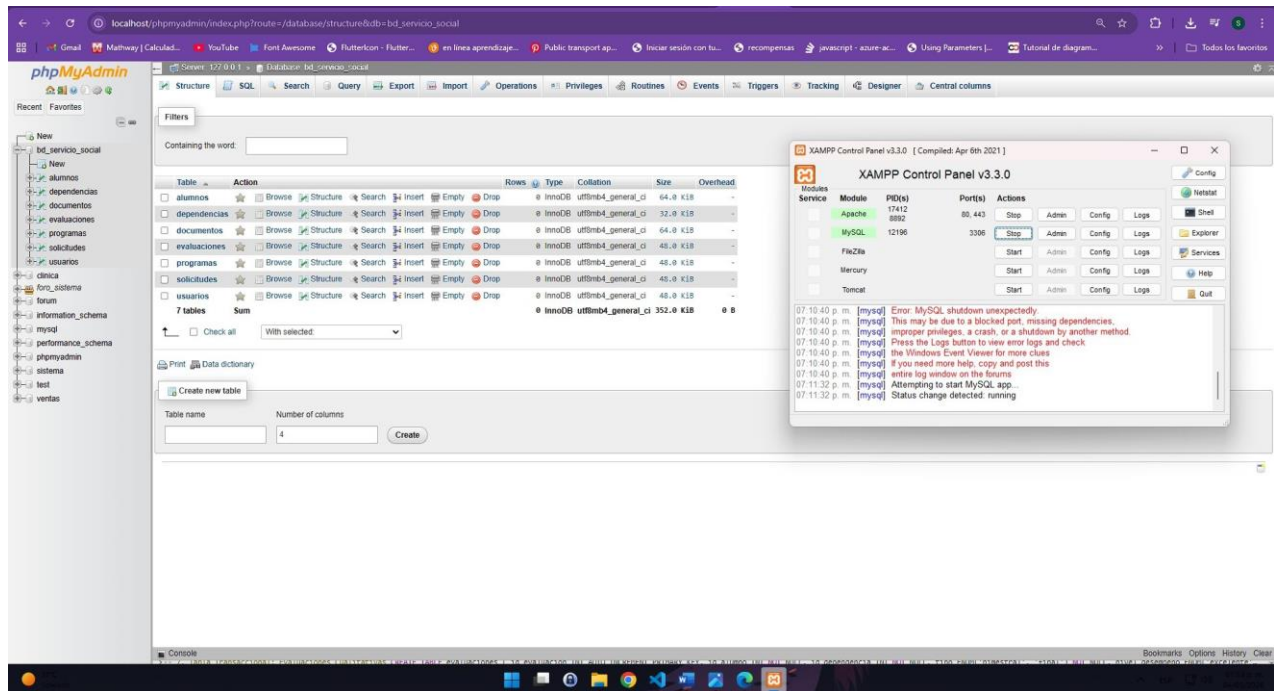
c) Mecanismos de seguridad, control de accesos e integridad referencial

En cuanto a seguridad técnica y la protección de datos confidenciales de los alumnos, el SGBD implementa un sistema robusto de privilegios y control de accesos basado en roles a nivel de servidor. Esto permite restringir las operaciones de lectura y escritura directamente en el esquema de la base de datos (bd_servicio_social), asegurando que solo los scripts autenticados del backend puedan realizar modificaciones. Asimismo, el motor InnoDB gestiona de forma nativa la integridad referencial mediante la declaración explícita de llaves primarias y llaves foráneas. Esto proporciona al sistema restricciones automatizadas en cascada o de restricción estricta, impidiendo, por ejemplo, que se elimine el registro de una dependencia externa si esta tiene alumnos o programas activos vinculados, manteniendo la base de datos limpia de registros huérfanos.

2.2 Diseño Lógico de la Base de Datos



2.3 Diseño Físico de la Base de Datos generado por el Manejador



3. CONSTRUCCIÓN DEL SISTEMA

3.1 Definición del Estilo de Programación

a) Modularidad y acoplamiento débil mediante Programación Orientada a Objetos

El estilo de programación se basa en el paradigma de la Programación Orientada a Objetos, estructurado de forma modular para reflejar la descomposición del sistema (Controlador principal, subsistemas operativos y utilitarios). Cada módulo documentado en el diseño (como el Modulo_Login, Modulo_Inscripcion o Modulo_Seguimiento) se programará como una clase independiente con responsabilidades únicas y bien definidas. Se priorizará un diseño de acoplamiento débil y alta cohesión, lo que significa que las clases interactuarán entre sí mediante interfaces y métodos públicos claros, ocultando la lógica interna de sus datos. Esto permitirá que modificaciones en las reglas de negocio de un módulo no provoquen efectos secundarios ni corrompan el funcionamiento de los componentes utilitarios compartidos, como lo son los gestores de archivos binarios o el sistema de notificaciones SMTP.

b) Estándares de codificación, nomenclatura y legibilidad del código

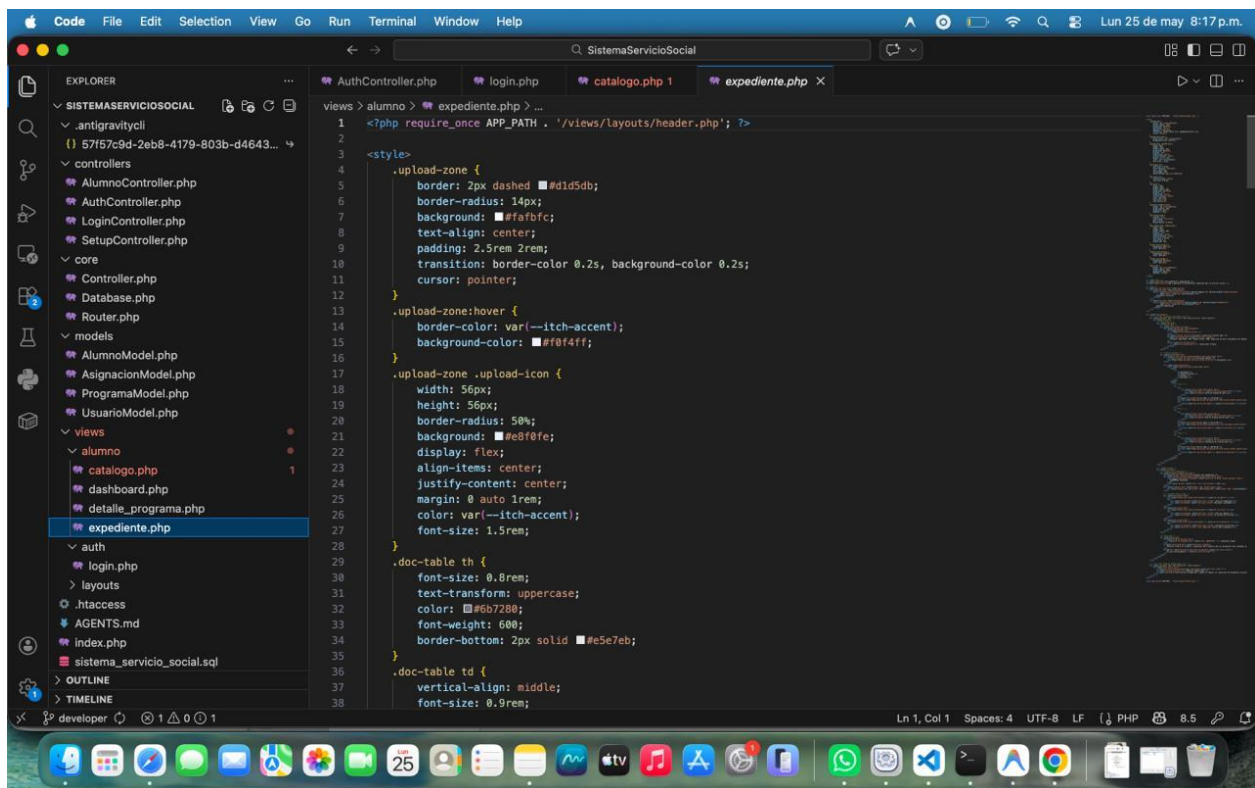
Para garantizar que el software sea legible, mantenible y uniforme para todos los integrantes del equipo, se adoptarán de forma estricta los estándares de recomendación de desarrollo para PHP, específicamente las pautas de estilo de código PSR (PHP Standard

Recommendations). Toda la nomenclatura del proyecto seguirá convenciones preestablecidas: los nombres de las clases utilizarán el formato StudlyCaps (letras mayúsculas iniciales), mientras que los métodos, funciones y variables emplearán el formato camelCase (primera letra minúscula y mayúsculas en palabras subsecuentes). Las estructuras de control de flujo, como las condicionales y los ciclos de validación de expedientes, mantendrán una indentación consistente de cuatro espacios por nivel y el uso obligatorio de llaves, eliminando códigos ambiguos o compactos. Estas reglas aseguran que cualquier fragmento de código escrito por un miembro del equipo pueda ser interpretado y auditado inmediatamente por otro sin problemas de comprensión.

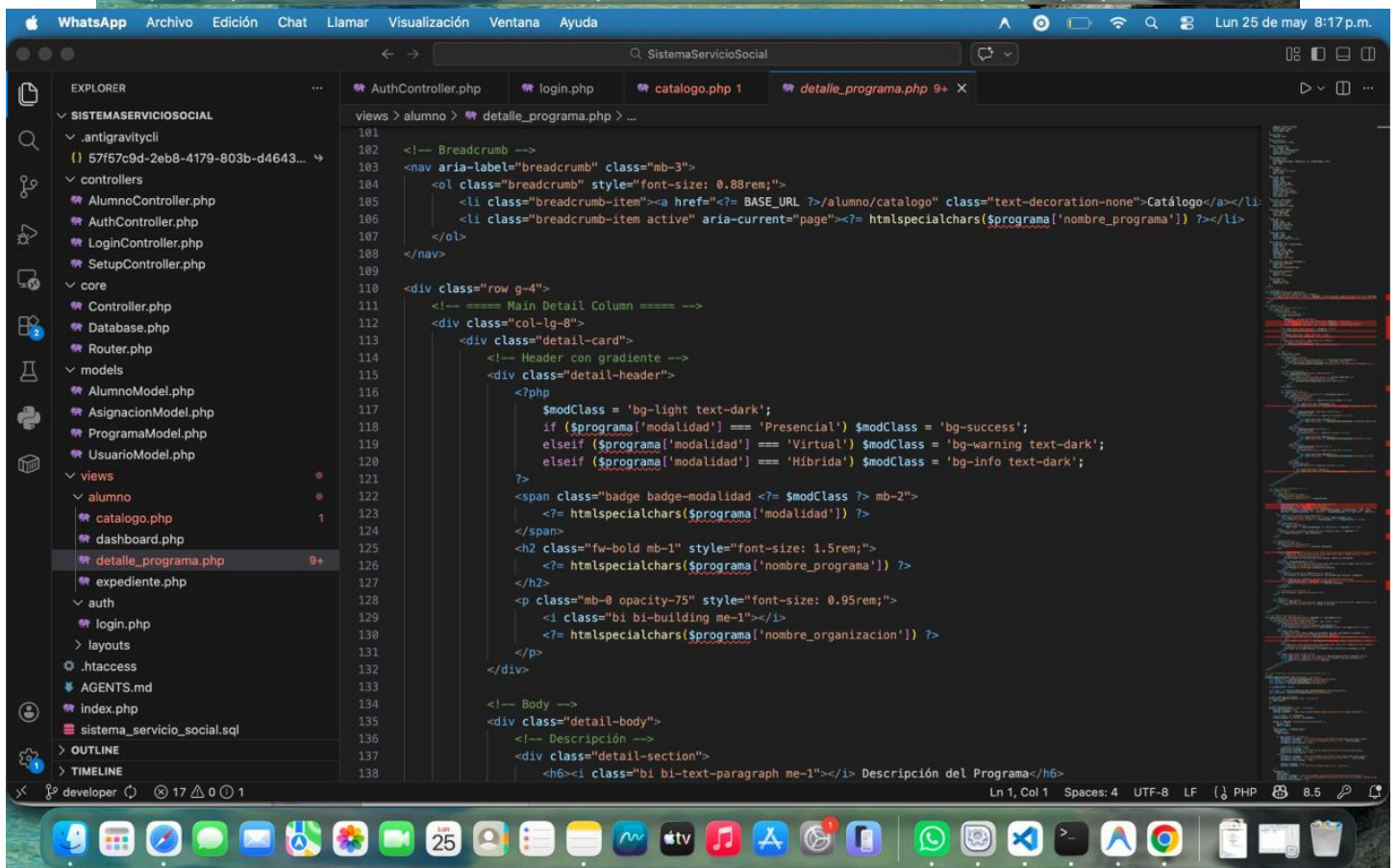
c) Programación defensiva y manejo estructurado de excepciones

El software implementará una filosofía de programación defensiva, asumiendo que los datos de entrada provenientes de los clientes (como formularios de dependencias o subidas de PDFs) pueden contener errores o malformaciones de forma intencional o accidental. En lugar de permitir que el intérprete de PHP falle de forma nativa o exponga rutas del servidor físico, toda la lógica transaccional y de comunicación estará envuelta en bloques estructurados de captura de errores (try-catch). El código estará diseñado para anticipar escenarios de falla, como la caída del servidor de correo institucional o la carga de un archivo corrupto. Al interceptar una excepción, el sistema ejecutará flujos alternos controlados, registrará la anomalía en una bitácora de errores interna y devolverá mensajes de retroalimentación claros y limpios hacia la interfaz de Bootstrap, manteniendo la estabilidad operativa del lado del usuario.

3.2 Código de Programación del Proyecto



```
1 <?php require_once APP_PATH . '/views/layouts/header.php'; ?>
2
3 <style>
4     .upload-zone {
5         border: 2px dashed #d5db;
6         border-radius: 14px;
7         background: #fafbfc;
8         text-align: center;
9         padding: 2.5rem 2rem;
10        transition: border-color 0.2s, background-color 0.2s;
11        cursor: pointer;
12    }
13    .upload-zone: hover {
14        border-color: var(--itch-accent);
15        background-color: #f0f4ff;
16    }
17    .upload-zone .upload-icon {
18        width: 56px;
19        height: 56px;
20        border-radius: 50%;
21        background: #e8f0fe;
22        display: flex;
23        align-items: center;
24        justify-content: center;
25        margin: 0 auto 1rem;
26        color: var(--itch-accent);
27        font-size: 1.5rem;
28    }
29    .doc-table th {
30        font-size: 0.8rem;
31        text-transform: uppercase;
32        color: #6b7280;
33        font-weight: 600;
34        border-bottom: 2px solid #e5e7eb;
35    }
36    .doc-table td {
37        vertical-align: middle;
38        font-size: 0.9rem;
```



```
101 <!-- Breadcrumb -->
102 <nav aria-label="breadcrumb" class="mb-3">
103     <ol class="breadcrumb" style="font-size: 0.88rem;">
104         <li class="breadcrumb-item"><a href="<= BASE_URL ?>/alumno/catalogo" class="text-decoration-none">Catálogo</a></li>
105         <li class="breadcrumb-item active" aria-current="page"><?= htmlspecialchars($programa['nombre_programa']) ?></li>
106     </ol>
107 </nav>
108
109 <div class="row g-4">
110     <!-- ===== Main Detail Column ===== -->
111     <div class="col-lg-8">
112         <div class="detail-card">
113             <!-- Header con gradiente -->
114             <div class="detail-header">
115                 <?php
116                     $modClass = 'bg-light text-dark';
117                     if ($programa['modalidad'] === 'Presencial') $modClass = 'bg-success';
118                     elseif ($programa['modalidad'] === 'Virtual') $modClass = 'bg-warning text-dark';
119                     elseif ($programa['modalidad'] === 'Hibrida') $modClass = 'bg-info text-dark';
120                 ?>
121                 <span class="badge badge-modalidad <?= $modClass ?> mb-2">
122                     <?= htmlspecialchars($programa['modalidad']) ?>
123                 </span>
124                 <h2 class="fw-bold mb-1" style="font-size: 1.5rem;">
125                     <?= htmlspecialchars($programa['nombre_programa']) ?>
126                 </h2>
127                 <p class="mb-0 opacity-75" style="font-size: 0.95rem;">
128                     <i class="bi bi-building me-1"></i> <?= htmlspecialchars($programa['nombre_organizacion']) ?>
129                 </p>
130             </div>
131             <!-- Body -->
132             <div class="detail-body">
133                 <!-- Descripción -->
134                 <div class="detail-section">
135                     <h6><i class="bi bi-text-paragraph me-1"></i> Descripción del Programa</h6>
```